

머신러닝 프로그래밍

Machine Learning Programming

Lecture 10-1 : 앙상블



충북대학교
CHUNGBUK NATIONAL UNIVERSITY



충북대학교
산업인공지능연구센터

앙상블의 이해

앙상블(Ensemble)

1. 사전적 의미: "오케스트라의 합주"

원래 프랑스어로 '함께', '조화', '통일체'를 뜻하는 단어

2. 머신러닝에서의 의미: "집단 지성 (Wisdom of the Crowd)"

여러 개의 평범한 모델(Weak Learner)을 모아서, 하나의 완벽에 가까운 강력한 모델(Strong Learner)을 만드는 기법

앙상블의 이해

단계별 데이터 분석 알고리즘 적용 전략

1단계: 베이스라인 모델 구축 (선형/로지스틱 회귀)

- 가장 대중적이며 결과 해석력이 뛰어난 알고리즘.
- 변수 간의 직관적 인과관계 파악 및 분석 성능의 기준점(Baseline)으로 활용.

2단계: 비선형성 및 복잡도 극복 (의사결정 트리 및 앙상블)

- 선형 모델로 설명 불가능한 복잡한 패턴 학습 시 적용.

3단계: 고차원 및 비정형 데이터 처리 (딥러닝)

- 다변량 시계열(Multivariate Time-series) 센서 데이터, 비전 데이터 등 극도로 복잡한 패턴 인식에 투입.

앙상블의 이해

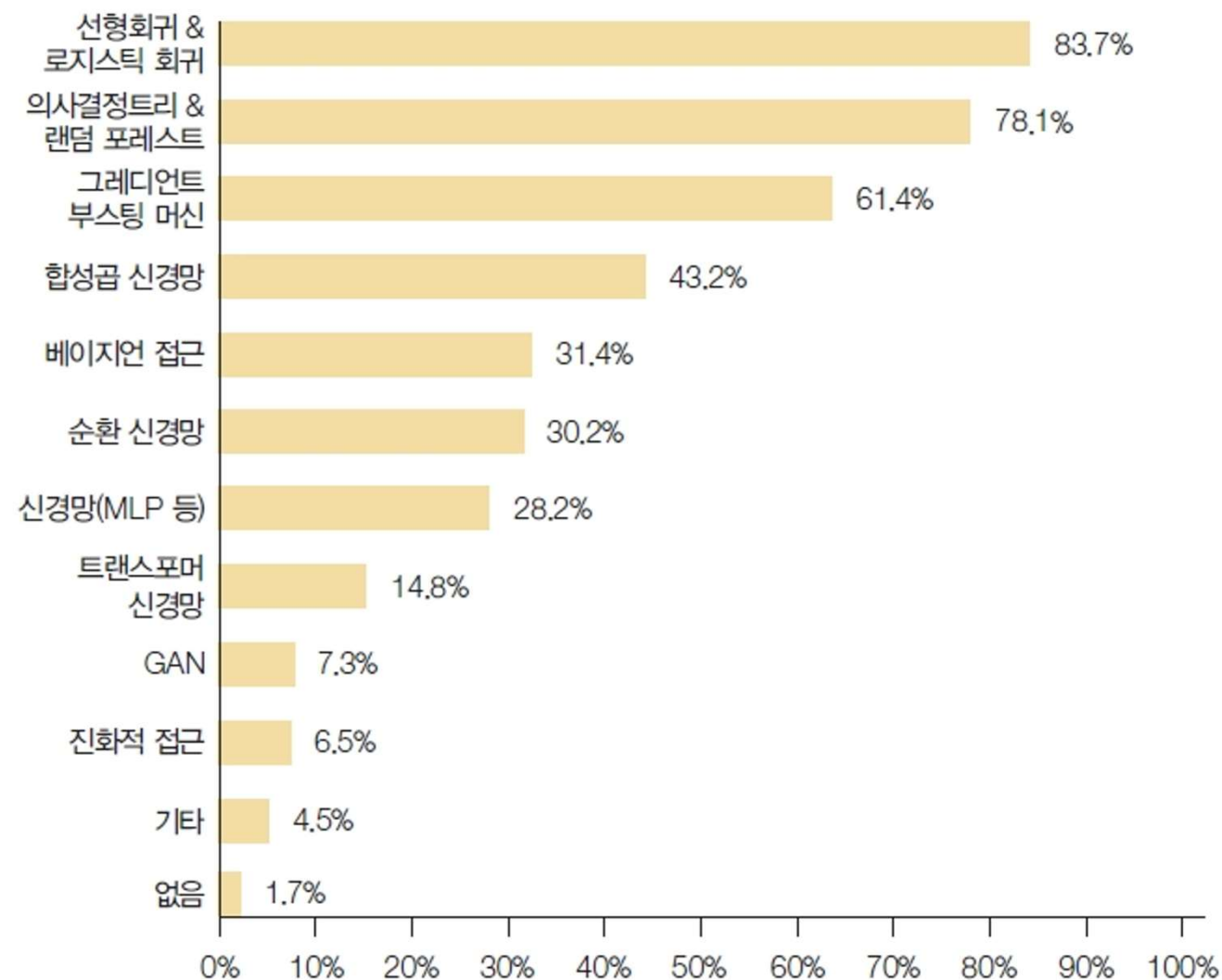


그림 1. 데이터 분석 시 사용하는 알고리즘 설문조사

앙상블의 이해

앙상블(Ensemble)의 목적

- 개별 모델이 가질 수 있는 약점(과적합, 편향 등)을 상호 보완하여 전체적인 예측의 안정성과 정확도를 극대화하는 것.\

메타 분류기 (Meta-Classifier)로서의 앙상블

- '메타(Meta)'란 상위 차원, 또는 추상화된 개념.
- 즉, 기초가 되는 여러 하위 분류기(Base classifiers)들을 지휘하고 종합하여 최종 결정을 내리는 '상위 레벨의 단일 분류기'라는 의미에서 앙상블 모델을 메타 분류기라고 명명함.

실무적 트레이드오프 (Trade-off)

- **단점:** 여러 개의 모델을 동시에 학습시키고 결과를 취합해야 하므로, 단일 모델 대비 **연산 자원 소모가 크고 학습 시간이 길다.**
- **장점:** 시간적 비용을 감수할 만큼 **압도적이고 안정적인 예측 성능**을 보장하기 때문에, 현업 데이터 분석이나 캐글(Kaggle) 같은 데이터 과학 대회에서 최종 모델로 가장 널리 쓰임.

앙상블의 이해_핵심 전략

앙상블 학습

보팅 (Voting) : 이기종 모델의 결합

- 학습 방식: 서로 다른 알고리즘(로지스틱, SVM 등)에 **동일한 데이터** 학습
- 결합 방식: 다수결(Hard Voting) 또는 가중 평균(Soft Voting)으로 단순 취합
- 특징: 가장 기본적이고 직관적인 앙상블 형태, 구현이 매우 쉬움

배깅 (Bagging) : 병렬 학습과 다양성

- 학습 방식: 복원 추출(Bootstrap)로 생성한 서브 데이터셋을 **독립적·병렬적** 학습
- 효과: 분산(Variance)을 줄여주어 **강건성(Robustness)** 확보
- 대표 알고리즘: 랜덤 포레스트 (Random Forest)

부스팅 (Boosting) : 순차 학습과 오답 노트

- 학습 방식: 전 모델이 틀린 데이터에 가중치 부여, 후 모델이 **순차적** 약점 보완
- 효과: 오차 극단적으로 줄여 편향(Bias) 감소. 정형 데이터에서 **정확도 우수**
- 대표 알고리즘: XGBoost, LightGBM, Gradient Boosting

스태킹 (Stacking) : 메타 모델의 재학습

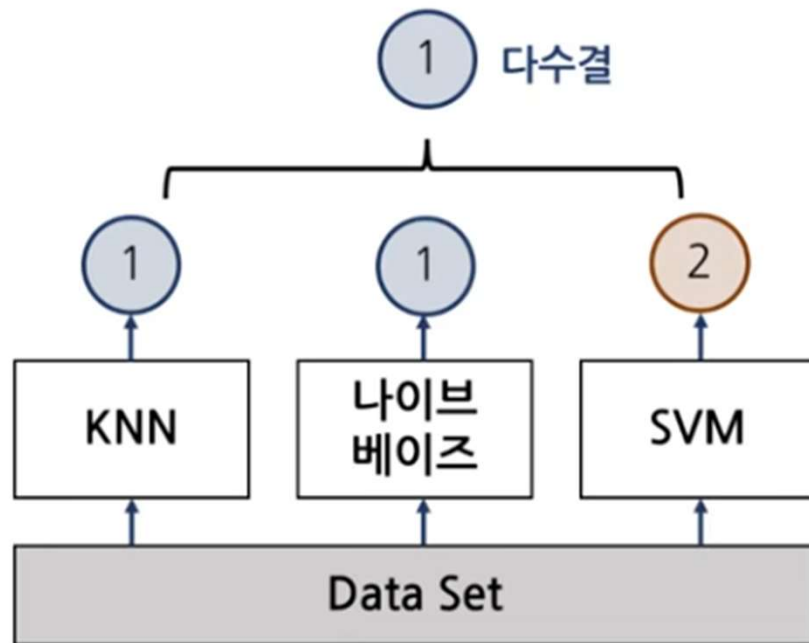
- 학습 방식: 하위 모델들의 예측 결과를 새로운 학습 데이터로 **삼아 재학습**
- 특징: "A모델이 틀릴 때 B모델의 의견을 따르는 게 맞다"는 패턴 자체를 메타 모델이 스스로 학습
- 활용: 캐글(Kaggle) 등 데이터 분석 대회에서 주로 사용

앙상블의 이해_핵심 전략

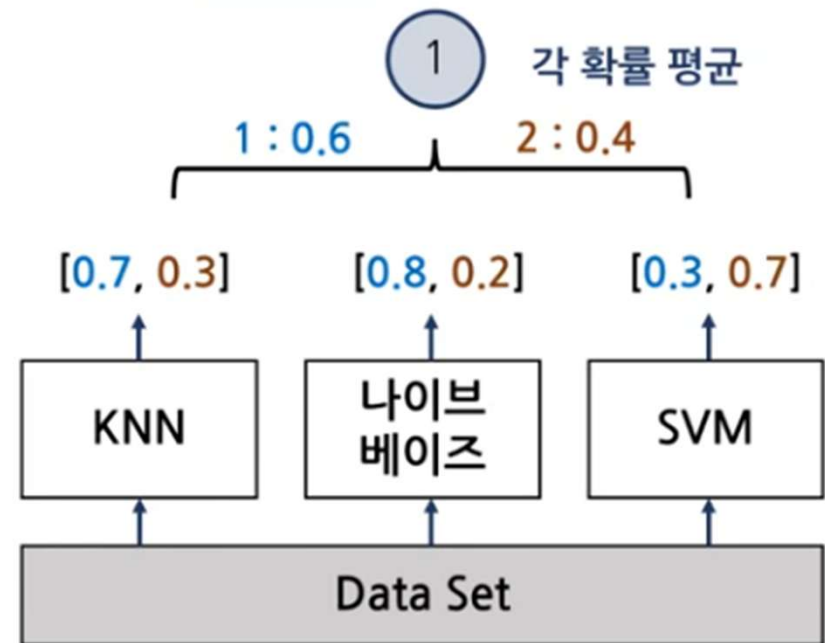
Voting

서로 다른 모델을 활용하여 예측한 값을 다수결 혹은 평균으로 최종결과 선정

Hard Voting



Soft Voting



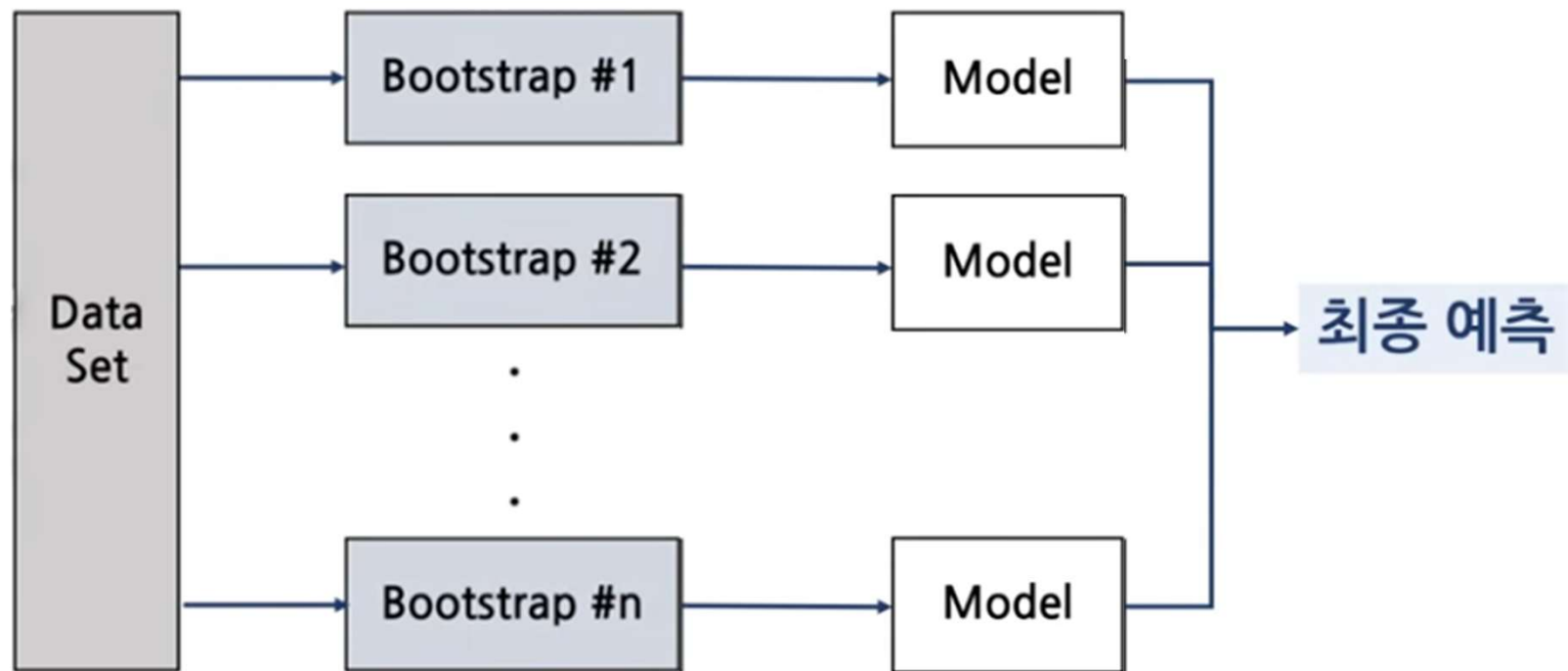
일반적으로 Soft Voting의 성능이 좋음

앙상블의 이해_핵심 전략

Bagging

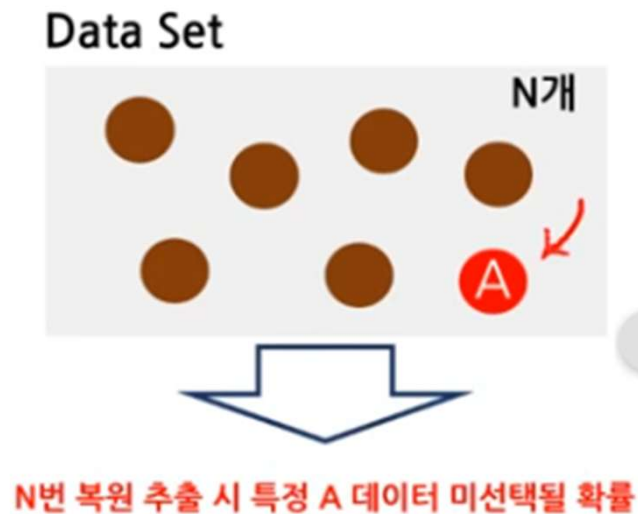
Bootstrap된 데이터를 활용하여 동일한 모델로 데이터를 학습하여 예측

* Bootstrap : 모집단으로부터 데이터를 복원 추출하는 샘플링 방식



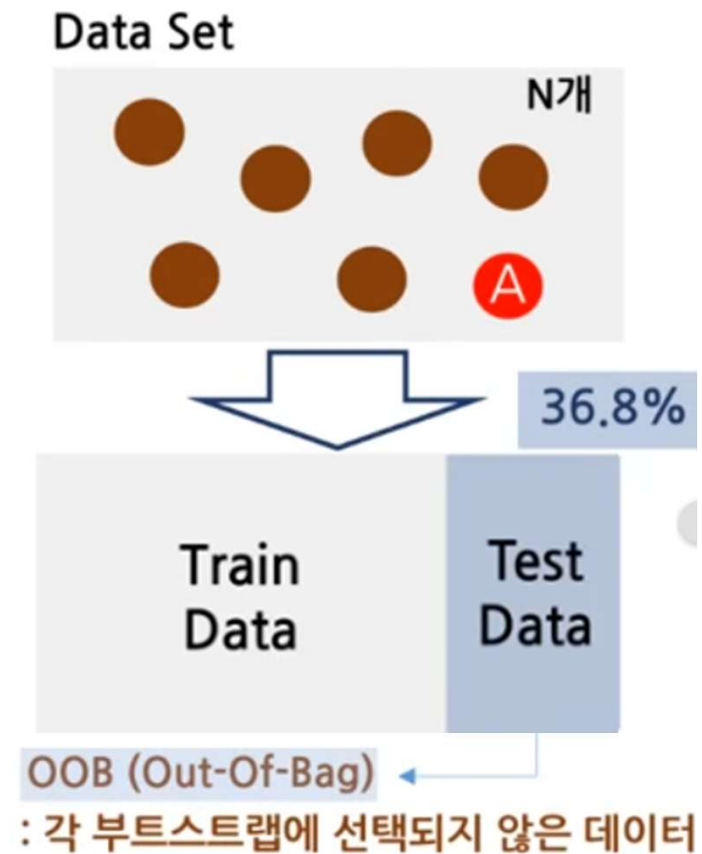
앙상블의 이해_핵심 전략

Bagging



$$P = \left(1 - \frac{1}{N}\right)^N$$

$$\lim_{N \rightarrow \infty} \left(1 - \frac{1}{N}\right)^N = 0.368$$



배깅과 랜덤 포레스트

약분류기와 강분류기로 알아보는 배깅 기법

- 배깅 기법은 여러 개의 약분류기(weak learner)로 강분류기(strong learner)를 만드는 것이다.
- 약분류기는 기본적으로 과소적합이 다소 있지만 과적합되어 있지 않은 모델. 다소 느슨하게 경계를 생성하는 여러 개의 약한 분류기를 앙상블한다면 좀 더 정확한 경계를 생성한다.
- 각각의 작은 데이터로는 모든 구체적인 분류 영역을 정할 수 없지만 많은 데이터로 투표한다면 더 높은 성능을 기대할 수 있다.

배깅과 랜덤 포레스트

부트스트래핑(Bootstrapping)

- 제한된 '원본 학습 데이터' 하나만으로, 조금씩 다른 여러 개의 서브 데이터셋을 무한히 생성해 내는 샘플링 기법.
- 배깅(Bagging) 기법이 여러 모델을 다양하게 학습시킬 수 있도록 데이터의 재료를 공급하는 역할.

복원 추출 (Sampling with Replacement)

- 데이터를 하나 뽑아 기록한 뒤, 그 데이터를 다시 전체 데이터 주머니에 집어넣고(복원) 섞은 후 다음 데이터를 뽑는 방식
- 원본과 동일한 크기(N개)의 데이터셋을 만들면, 새로 만들어진 '학습 데이터셋 1' 안에는 특정 데이터가 여러 번 중복되어 들어갈 수도 있고, 반대로 한 번도 뽑히지 않는 데이터가 발생할 수도 있음.

OOB 데이터

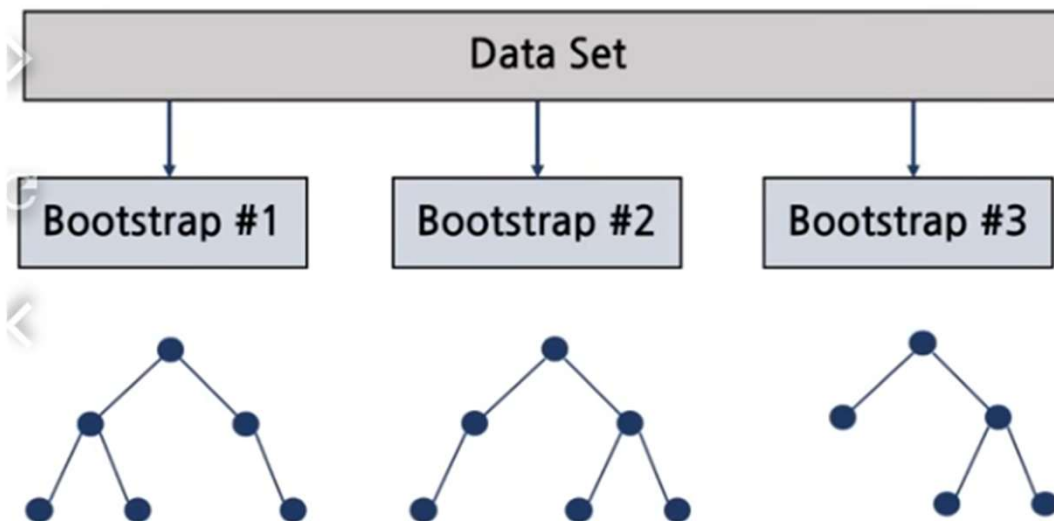
- 수학적으로 복원 추출을 통해 한 번도 뽑히지 않을 확률을 계산해 보면, 전체 원본 데이터의 약 36.8%.
- 나중에 모델의 성능을 평가하는 훌륭한 자체 테스트 데이터로 활용.

앙상블의 이해_핵심 전략

Bagging

랜덤포레스트(RandomForest)

여러 개의 무작위로 생성된 결정트리를 활용한 배깅 방식의 알고리즘

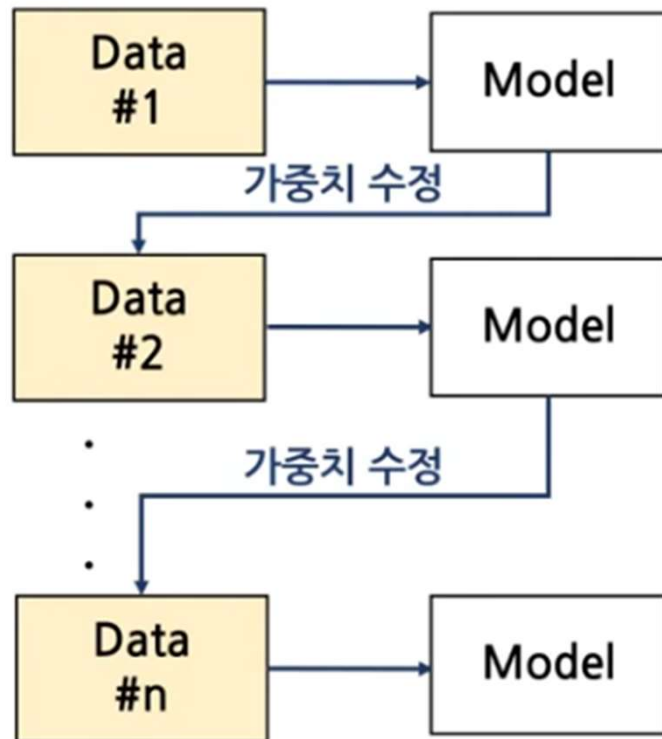


- 트리 노드 분할 시 무작위성 주입
: 극단적 무작위 랜덤 포레스트인 경우
→ 익스트라 트리
- 내부 구조를 알 수 없는 블랙박스 모델
- 편향을 손해 보는 대신 분산을 낮춤
: Overfitting에 강력한 모델
- 많은 메모리를 소모하지만 빠른 속도

앙상블의 이해_핵심 전략

Boosting

이전 모델에서 학습된 결과에 따라 가중치로 데이터를 개선하여 다음 모델에 **순차적**으로 영향



AdaBoost(Adaptive Boosting)

과소적합했던 훈련 샘플의 가중치를 높여가는 방법

GBM(Gradient Boosting Machine)

이전 모델에서 만든 오차에 대해 새로운 모델에서 학습하는 방법

XGBoost(eXtreme Gradient Boosting)

GBM보다 빠르고 규제 기능이 포함되어 있어 과대적합 방지 가능

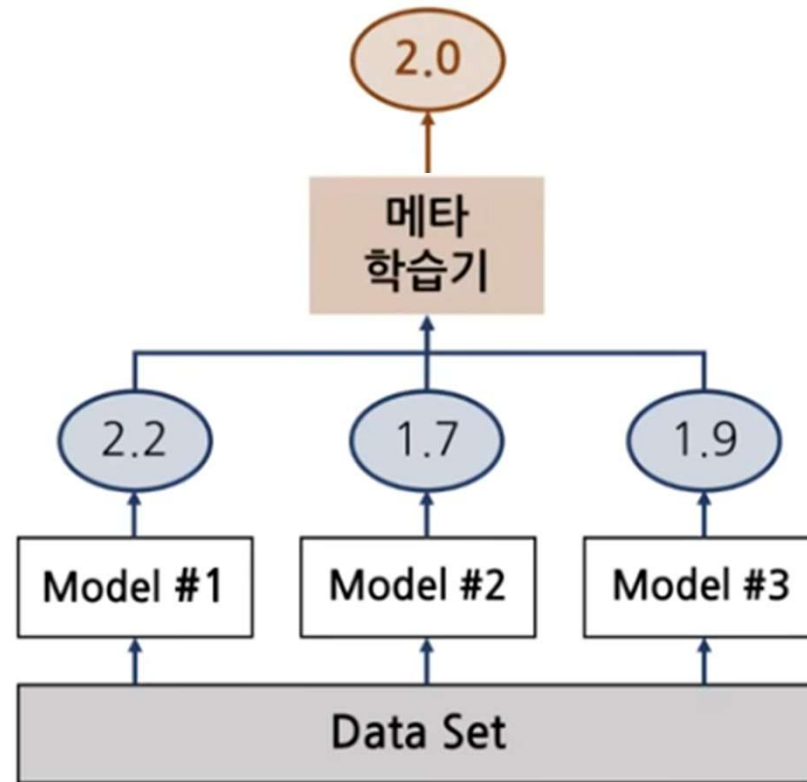
Light GBM

GBM의 느린 학습 속도를 개선하기 위하여 경량화

앙상블의 이해_핵심 전략

Stacking

각각의 모델에서 학습한 예측 결과를 학습하여 최종 결과를 도출



실습

< 비교 >

1. max_depth
2. 2. rf = RandomForestClassifier(n_estimators=100)
3. 3. make_moons(noise=0.55)

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.datasets import make_moons
4 from sklearn.model_selection import train_test_split
5 from sklearn.tree import DecisionTreeClassifier
6 from sklearn.ensemble import (RandomForestClassifier, GradientBoostingClassifier,
7                               VotingClassifier, StackingClassifier)
8 from sklearn.linear_model import LogisticRegression
9 from sklearn.svm import SVC
10 from sklearn.neighbors import KNeighborsClassifier
11 from sklearn.metrics import accuracy_score
12
13 # 1. 극한 노이즈 데이터 생성
14 X, y = make_moons(n_samples=500, noise=0.55, random_state=42)
15 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
16
17 # 2. 모델 세팅 (인라인 선언으로 변수 최소화)
18 dt = DecisionTreeClassifier(random_state=42)
19 rf = RandomForestClassifier(n_estimators=100, random_state=42)
20 gb = GradientBoostingClassifier(n_estimators=100, max_depth=5, random_state=42)
21
22 models = {
23     'Single Tree': dt,
24     'Voting': VotingClassifier([('lr', LogisticRegression()), ('dt', dt),
25                               ('svc', SVC(probability=True, random_state=42))], voting='soft'),
26     'Bagging (RF)': rf,
27     'Boosting (GBM)': gb,
28     'Stacking': StackingClassifier([('rf', rf), ('gb', gb), ('knn', KNeighborsClassifier())],
29                                   final_estimator=LogisticRegression())
30 }
31
32 # 3. 모델 학습, 평가 및 시각화 (반복문 통합)
33 print("\n [극한 노이즈 환경] 앙상블 모델 성능 리포트\n ")
34 fig, axes = plt.subplots(1, 5, figsize=(25, 4))
35 xx, yy = np.meshgrid(np.linspace(X[:, 0].min() - 0.5, X[:, 0].max() + 0.5, 100),
36                      np.linspace(X[:, 1].min() - 0.5, X[:, 1].max() + 0.5, 100))
37
38 for ax, (name, model) in zip(axes, models.items()):
39     model.fit(X_train, y_train)
40     acc = accuracy_score(y_test, model.predict(X_test))
41
42     # 텍스트 결과 출력 (간결하게 한 줄로)
43     print(f"> {name:15s} 정확도: {acc * 100:.2f}%")
44
45     # 그래프 시각화
46     Z = model.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
47     ax.contourf(xx, yy, Z, alpha=0.3, cmap='coolwarm')
48     ax.scatter(X_train[:, 0], X_train[:, 1], c=y_train, cmap='coolwarm', edgecolors='k', s=20)
49
50     ax.set_title(f"{name}\nAcc: {acc:.3f}", fontsize=14, fontweight='bold', color='darkred')
51     ax.axis('off')
52
53 plt.tight_layout()
54 plt.show()
```